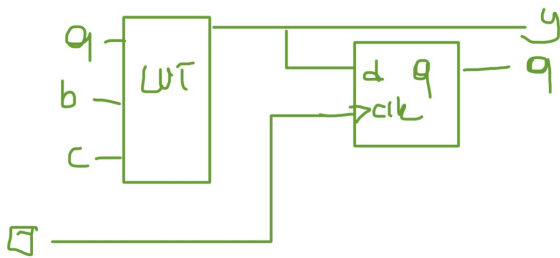
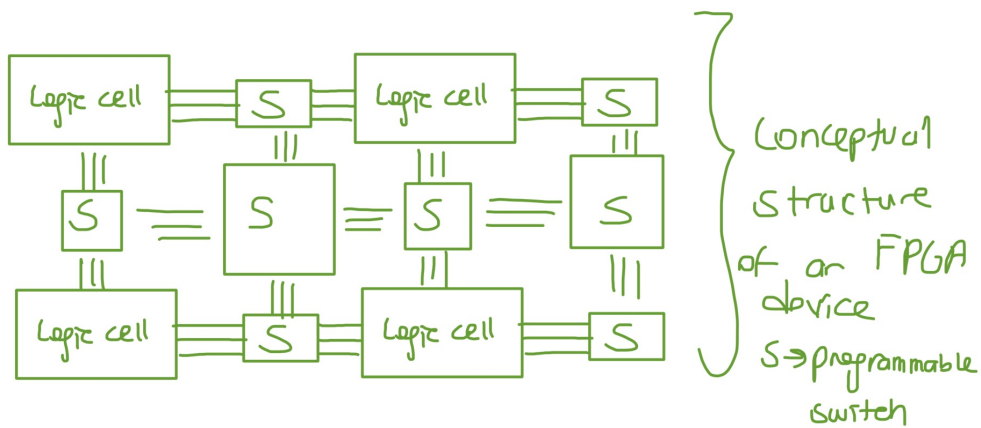


CPU; Sabit donanım, Sabit bağlantılar
High level languages - Compiler

FPGA: field programmable gate array



alanda değiştirebilir. Devre blokları değiştirilebilir.
Paralel mimari

Logic Design,

→ Basic building blocks for logic design are

→ Gate

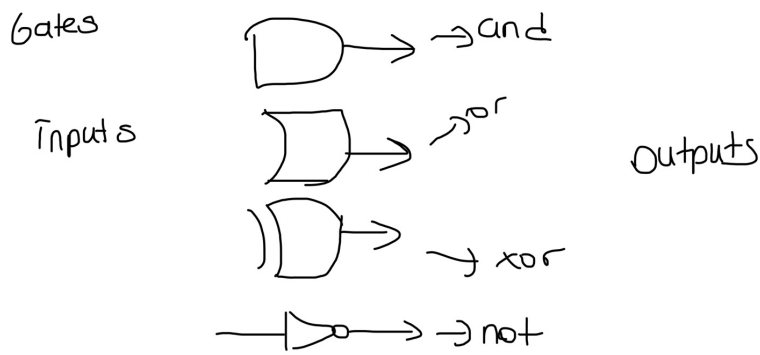
→ Muxes / Demuxes

→ Registers / Memory

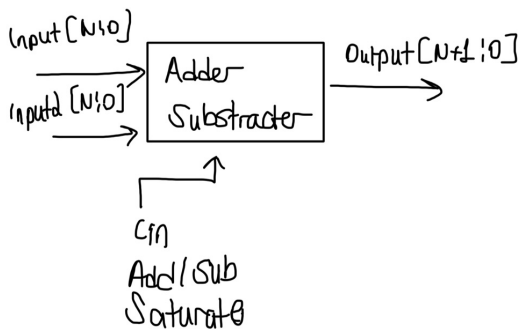
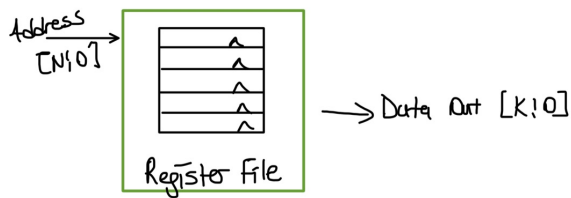
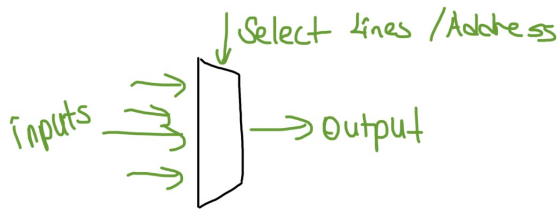
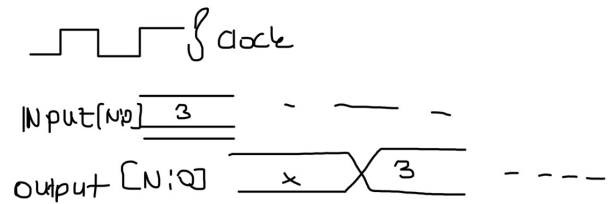
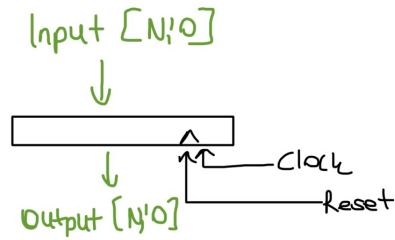
→ Arithmetic Operations

→ State machine

→ any combinations of above

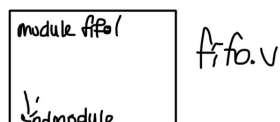


Register



Verilog :

A reasonable size replicatable block e.g. FIFO, counter and decrementer that we covered are good candidates.



```

module fifo (
  input [15:0] data-in,
  input push,
  input pop,
  output [15:0] data-out,
);
  // The code for modelling FIFO here
endmodule

```

```

module counter (
  input increment,
  input decrement,
  output [3:0] count

```

```
);
```

```

  counter counter_inst1 (
    .increment (push),
    .decrement (pop),
    .count (count)
  );
  // hide the ports

```

```
begin
```

```
  case (addr)
```

```
    2'd0 : out = a;
```

```
    2'd1 : out = b;
```

```
    2'd2 : out = c;
```

```
    2'd3 : out = d;
```

```
  endcase
```

```
end
```

```
/
```

```
module traffic (
```

```
    input clk, rst;
```

```
    output [2:0] result
```

```
); reg [23:0] counter
   typedef enum reg[1:0] { S_RED = 2'b00, S_YELLOW = 2'b01, S_GREEN = 2'b10 } state_t
```

```
state_t state, next_state;
```

```
always @(*) begin
```

```
    case (state)
```

```
        S_RED: next_state = (counter == 24'd 10.000.000) ? S_GREEN : S_RED;
```

```
        S_GREEN: next_state = (counter == 24'd 7.000.000) ? S_YELLOW : S_GREEN;
```

```
        S_YELLOW: next_state = (counter == 24'd 3.000.000) ? S_RED : S_YELLOW;
```

```
        default: next_state = S_RED;
```

```
    end case
```

```
end
```

```
always @ (posedge clk, posedge rst) begin
```

```
    if (rst) begin
```

```
        state <= S_RED;
```

```
        counter <= 0;
```

```
    end else begin
```

```
        if (state == next_state)
```

```
            counter <= counter + 1;
```

```
        else
```

```
            counter <= 0;
```

```
        state <= next_state;
```

```
    end
```

```
end
```

```
always (*) begin
```

```
    case (state)
```

```
        S_RED: results = 3'b100;
```

```
        S_YELLOW: results = 3'b010;
```

```
        S_GREEN: results = 3'b001;
```

```
        default: results = 3'b100;
```

```
    end case
```

```
end
```

```
endmodule
```